CS224N | Word2Vec Model

Selene

August 3, 2026

1 Word Representation

1.1 Signifier and Signified

A word is a signifier. It represents a signified object in the real or imagined world. A signifier, or the corresponding signified object, can refer to something very broad, such as uncle and drink, but it can also refer to something much more specific, such as uncle Zuko and coffee. Therefore, word meaning is extremely important, and even a small change may have a large influence.

1.2 Independent Words and Vectors

The simplest way to represent words is to treat them as mutually independent and unrelated entities. Mutually independent components are usually represented by one-hot vectors, that is, standard basis vectors. For example,

$$v_{\text{tea}} = [0, 0, 1, 0, \dots, 0]^T, \qquad v_{\text{coffee}} = [0, 0, 0, 0, \dots, 1, \dots, 0]^T.$$

However, the problem with one-hot representations is that they cannot encode any similarity relation. The dot product of any two different vectors is 0. In reality, however, words in a sentence are related to each other. Therefore, we introduce the next approach.

1.3 Vectors from Manually Annotated Discrete Attributes

This approach manually constructs a feature vector whose entries describe the attributes of a word. Clearly, this is not a good solution.

2 Distributional Semantics and Word2Vec

2.1 Distributional Hypothesis

The distributional hypothesis states that the meaning of a word can be inferred from the distribution of contexts in which it appears. This is one of the most influential ideas in NLP.

2.2 Co-occurrence Matrix and Document Context

To implement the idea of the distributional hypothesis, we can construct a co-occurrence matrix, denoted by X. The size of this matrix is $|V| \times |V|$, where X_{ij} denotes the number of times word i and word j appear in the same context.

In a sentence, we can mark multiple windows of different sizes. Short windows tend to encode syntactic properties, while long windows tend to encode semantic properties or even topic-level properties.

The problem with this approach is that high-dimensional vectors are difficult to handle in modern neural systems, and high-frequency words receive too much weight.

2.3 Word2Vec Model and Objective Function

Word2Vec represents each word in a fixed vocabulary as a low-dimensional vector whose dimension is much smaller than the vocabulary size. It learns the values of word vectors so that they can be used to predict the distribution of context words through a simple function. The model introduced here is **Skip-gram Word2Vec**.

Skip-gram Word2Vec

Let C, O be a pair of unknown random variables, where $C \in V$ denotes the center word, and $O \in V$ denotes an outside word appearing in the context of the center word. We use c, o to denote their concrete values.

Suppose the vocabulary is V. Each word has two d-dimensional vector representations:

- u_w: the vector of word w when it is used as a context word;
- v_w: the vector of word w when it is used as a center word.

We arrange all word vectors into matrices

$$U, V \in \mathbb{R}^{|V| \times d}.$$

Given center word c, the probability that the context word is o is defined as

$$p_{U,V}(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}.$$

Here, $u_o^\top v_c$ represents the similarity between two word vectors. The larger the dot product is, the more likely the model believes that word o appears in the context of word c.

Softmax converts the dot-product scores of all words into a probability distribution:

$$p_{U,V}(\cdot \mid c) \in \mathbb{R}^{|V|}.$$

The goal of training Word2Vec is to make the predicted probability distribution close to the true distribution of context words for word c in the corpus, namely the row of the word co-occurrence matrix corresponding to c.

So far, we have only defined the model. We still need to determine the parameters U, V through training. Word2Vec uses the cross-entropy loss to make the predicted conditional distribution close to the true context distribution:

$$\min_{U,V} \mathbb{E}_{o,c} \left[-\log p_{U,V}(o \mid c) \right].$$

Here, (o, c) denotes a context word and center word pair sampled from the corpus. For each sample, the model computes the negative log probability of the true context word o given the center word c:

$$-\log p_{U,V}(o \mid c).$$

If the model assigns a larger probability to the true context word, the loss becomes smaller. Therefore, the training process continuously adjusts U, V to increase the predicted probability of real word pairs.

2.4 Estimating Word2Vec from a Corpus

Empirical Loss of Word2Vec

Let D be a collection of documents $\{d\}$. Each document is a word sequence $w_1^{(d)}, \ldots, w_m^{(d)}$, where $w^{(d)} \in V$. Let $k \in \mathbb{N}_{++}$ be a positive integer window size. We now connect the random variables C, O with this concrete dataset.

The center word C takes each word w_i in the document in turn. For each such w_i, the outside words are

$$\{w_{i-k}, \ldots, w_{i-1}, w_{i+1}, \ldots, w_{i+k}\}.$$

Therefore, the objective function becomes

$$L(U, V) = \sum_{d \in D} \sum_{i=1}^{m} \sum_{\substack{j=-k \\ j \neq 0}}^{k} -\log p_{U,V}\left(w_{i+j}^{(d)} \mid w_i^{(d)}\right).$$

Here, we sum over all documents, all words in each document, and all words in the window. In this way, we accumulate the negative log-likelihood of outside words given center words.

Gradient-based Estimation

We first make an initial guess with very little information for U, V, and then repeatedly move in the direction that locally improves this guess the most. This gives a gradient-based implementation.

For a scalar function f, its gradient with respect to the parameter matrix U is denoted by $\nabla_U f$. The gradient gives the direction in which the function increases fastest. Therefore, if we want to minimize the loss function, we should update the parameters in the opposite direction of the gradient.

Before training starts, the word vector matrices U, V are usually initialized as small random numbers:

$$U^{(0)}, V^{(0)} \sim N(0, 0.001)^{|V| \times d}.$$

In other words, each entry of the matrices is independently sampled from a normal distribution with mean 0 and small variance. Then gradient descent is used to update the parameters repeatedly. For example, the update formula for matrix U is

$$U^{(i+1)} = U^{(i)} - \alpha \nabla_U L\left(U^{(i)}, V^{(i)}\right).$$

Here, α is the learning rate, which controls the step size of each parameter update. Matrix V is updated in the same way:

$$V^{(i+1)} = V^{(i)} - \alpha \nabla_V L\left(U^{(i)}, V^{(i)}\right).$$

By continuously updating U, V in the direction that decreases the loss function, the predicted distribution of the model gradually approaches the true distribution of context words.

Stochastic Gradient

For now, we do not consider how to compute the gradient explicitly. Computing $L(U, V)$ is very expensive because it requires traversing the entire training set. Therefore, we use stochastic gradient optimization.

2.5 A Complete Gradient Derivation

First, we write down the gradient, and then use the linearity of differentiation to move the gradient operator into the summation:

$$\nabla_{v_c} \hat{L}(U, V) = \sum_{d \in D} \sum_{i=1}^{m} \sum_{\substack{j=-k \\ j \neq 0}}^{k} -\nabla_{v_c} \log p_{U,V}\left(w_{i+j}^{(d)} \mid w_i^{(d)}\right).$$

Now we rewrite $w_{i+j}^{(d)}$ as o, and rewrite $w_i^{(d)}$ as c:

$$\begin{aligned} \nabla_{v_c} \log p_{U,V}(o \mid c) &= \nabla_{v_c} \log \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)} \\ &= \nabla_{v_c} \log \exp(u_o^\top v_c) - \nabla_{v_c} \log \sum_{w \in V} \exp(u_w^\top v_c). \end{aligned}$$

We take the gradient of the two parts with respect to v_c separately.

The first part is

$$\nabla_{v_c} \log \exp(u_o^\top v_c) = \nabla_{v_c}(u_o^\top v_c) = u_o.$$

The second part is

$$\begin{aligned} \nabla_{v_c} \log \sum_{w \in V} \exp(u_w^\top v_c) &= \frac{\sum_{x \in V} \exp(u_x^\top v_c) u_x}{\sum_{w \in V} \exp(u_w^\top v_c)} \\ &= \sum_{x \in V} p_{U,V}(x \mid c) u_x. \end{aligned}$$

Therefore,

$$\nabla_{v_c} \log p_{U,V}(o \mid c) = u_o - \sum_{x \in V} p_{U,V}(x \mid c) u_x.$$

Here, u_o is the actually observed context word vector, while

$$\sum_{x \in V} p_{U,V}(x \mid c) u_x$$

is the expectation of the context word vector under the model's predicted distribution. Therefore, this gradient can be understood as observation minus expectation.

For the negative log-likelihood loss, the gradient has the opposite direction:

$$\nabla_{v_c} \left[-\log p_{U,V}(o \mid c) \right] = \sum_{x \in V} p_{U,V}(x \mid c) u_x - u_o.$$

Gradient descent makes the center word vector v_c closer to the true context word vector u_o, while pushing it away from word vectors to which the model incorrectly assigns high probabilities.